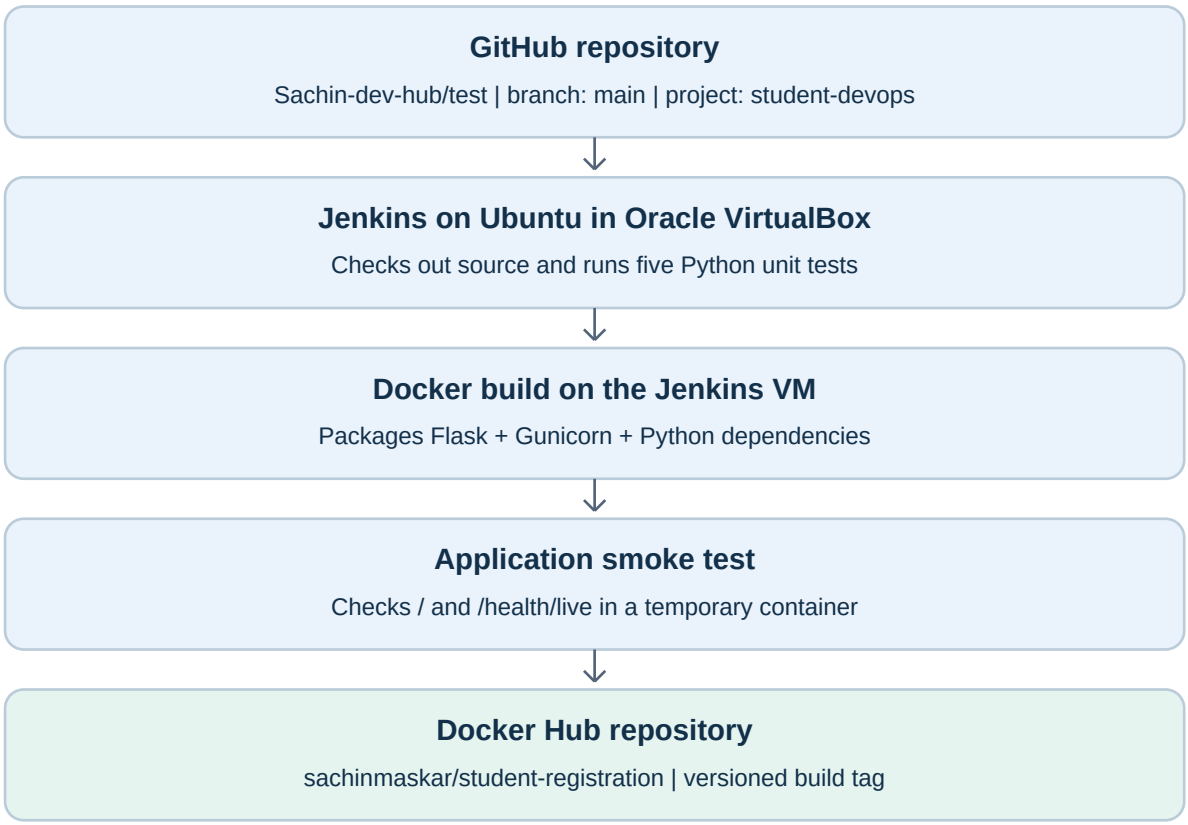
Student Registration DevOps Architecture

A practical guide to your application, Jenkins build pipeline, Docker Hub images, and Kubernetes deployment design.

1. Build and publish architecture - working



What is confirmed

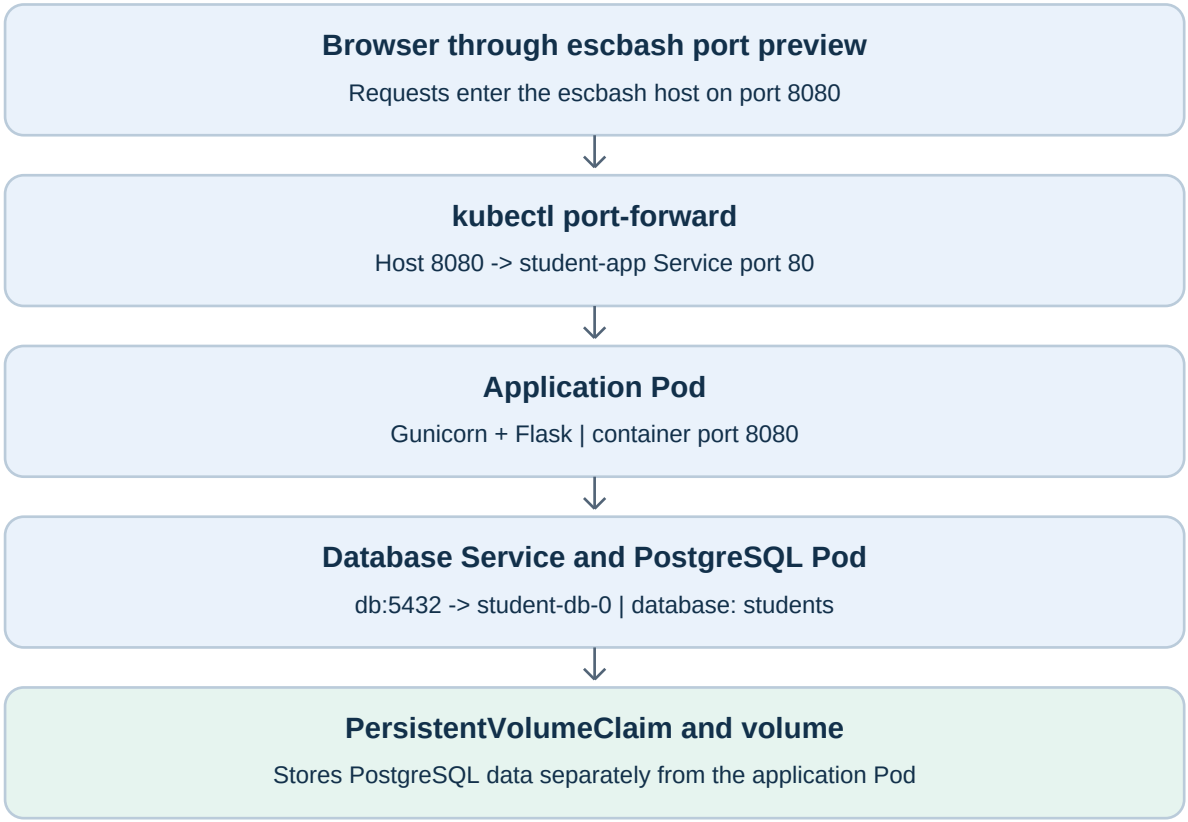
You confirmed that the pipeline now works after correcting the Dockerfile comments and Docker Hub token permissions. Jenkins builds on your Ubuntu VM; the application environment is on escbash.

Where the scope currently ends

This pipeline tests, builds, and publishes an image. It does not yet deploy automatically to Kubernetes. Treat it as CI with image publishing; automated deployment is the next addition.

How the application runs

The following is the intended Kubernetes layout from the supplied manifests. A successful Kubernetes deployment has not yet been confirmed in this conversation.



Application request flow

The browser loads HTML from GET /. JavaScript calls /api/students. Flask validates requests and uses Psycopg to execute parameterized SQL. PostgreSQL stores the records and Flask returns JSON to the browser.

Docker Compose version - already verified

In Compose, host port 8080 maps directly to application container port 8080. The app reaches PostgreSQL through the Compose service name db. A named Docker volume holds database files. There is no Kubernetes Service or kubectl port-forward in this route.

Separate data: Compose and Kubernetes have separate database volumes. Moving the image does not move student records. A database backup and restore is required to transfer them.

What each part does

Component	Responsibility
GitHub	Stores source, Dockerfile, Jenkinsfile, tests, Compose and Kubernetes YAML. It does not store live database records.
Jenkins	Loads student-devops/Jenkinsfile from main and executes each stage in order. A failed stage prevents later stages from running.
Docker image	Contains application code and dependencies. Tag format: build-BUILD_NUMBER-GIT_SHA. Existing v2 is a separate tag.
Gunicorn / Flask	Gunicorn runs two worker processes in one container. Flask handles pages, validation and CRUD endpoints.
PostgreSQL	Stores student names, unique email addresses, courses and creation times. Database port 5432 stays internal.
ConfigMap / Secret	ConfigMap holds non-secret settings. Secret supplies DB_PASSWORD. Do not commit .env or access tokens.
Persistent storage	Compose uses a named volume; Kubernetes uses a PVC. Neither replaces an off-host backup.

Health checks

/health/live: checks the application can respond without querying PostgreSQL. A database outage should not trigger repeated application liveness restarts.

/health/ready: queries the students table. Kubernetes uses readiness to withhold Service traffic from application Pods that are not ready.

Credentials and network boundaries

Jenkins credential ID **dockerhub** contains the Docker Hub username and a token with Read & Write permission. Login uses a temporary Docker configuration directory, removed when the step exits.

Jenkins needs outbound access to GitHub, Python packages and Docker Hub. For now it does not need access to the escbash Kubernetes API. Later, deployment requires a reachable API endpoint and a restricted Kubernetes identity.

Follow one release end to end

1. Source change

Edit application code, review the diff, stage, commit and push to main. If working on master, merge the pull request into main. Jenkins builds main.

```
git add app/
git commit -m "Update student registration application"
git push origin main
```

2. Build and publish

Start Build Now, or enable Poll SCM with H/5 * * * *. Jenkins checks out the commit, runs tests, builds the image, checks startup and pushes its generated tag. Poll SCM works without exposing your VirtualBox VM to GitHub.

The five unit tests and smoke test do not prove real PostgreSQL CRUD works. Add a disposable database integration test as the next testing improvement.

3. Deploy to Kubernetes - manual next step

Once the student-app Deployment and database exist, run these commands on escbash. Replace YOUR_NEW_TAG with the full successful Jenkins image tag:

```
kubectl -n student-devops set image deployment/student-app \
app=sachinmaskar/student-registration:YOUR_NEW_TAG
kubectl -n student-devops rollout status \
deployment/student-app --timeout=180s
```

The image must be publicly readable or the Pod needs an imagePullSecret. Verify registration, editing, deletion and record persistence after deployment.

4. Inspect or roll back

```
kubectl -n student-devops get pods,svc,pvc
kubectl -n student-devops logs deployment/student-app --tail=50
kubectl -n student-devops rollout history deployment/student-app
kubectl -n student-devops rollout undo deployment/student-app
```

Rollback needs an available previous revision and image. It restores a Pod template, not database records or schema. Restart port-forward if its selected Pod is replaced.

Explain it in an interview

"I built a Flask and PostgreSQL student registration lab. Jenkins checks out GitHub code, runs tests, builds a Docker image, checks startup and pushes a versioned tag to Docker Hub. I prepared Kubernetes manifests with Services, health probes and persistent storage. Automated Kubernetes deployment is the next phase."

Limits: one kind node does not provide node-level high availability. The lab has no login or HTTPS yet. Use sample data and export backups before a lab reset.